Expression Evaluation (Dependency Graph)

Problem Statement

Given equations like T1=2, T3=T1+T2, and a target variable, evaluate the target by resolving dependencies.

Variants

- Operators: + - * /

- Alia

How to Leverage the Solution

Build a graph and use DFS + memoization for all variants; add cycle/undefined checks once.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;

public final class ExpressionGraph {
 private final Map<String, Node> variableToNode = new HashMap<>();

 public static ExpressionGraph fromEquations(List<String> equations) {
  ExpressionGraph graph = new ExpressionGraph();
  for (String eq : equations) {
   graph.addEquation(eq);
  }
  return graph;
 }

 public double evaluate(String target) {
  Map<String, Double> memo = new HashMap<>();
  Set<String> visiting = new HashSet<>();
  return evaluateVar(target, memo, visiting);
 }

 private void addEquation(String equation) {
  String[] parts = equation.split("=");
  if (parts.length != 2) throw new IllegalArgumentException("Bad equation: " + equation);
  String lhs = parts[0].trim();
  String rhs = parts[1].trim();
  Node node = parseRhs(rhs);
  variableToNode.put(lhs, node);
 }

 private static Node parseRhs(String rhs) {
  String[] tokens = rhs.split("\\s+");
  if (tokens.length == 1) {
   Term term = parseTerm(tokens[0]);
   if (term.isConst) return Node.constant(term.value);
   else return Node.identity(term.variableName);
  } else if (tokens.length == 3) {
   Term left = parseTerm(tokens[0]);
   char op = tokens[1].charAt(0);
   Term right = parseTerm(tokens[2]);
   return Node.binary(left, op, right);
  }
```

```
 throw new IllegalArgumentException("Unsupported RHS: " + rhs);
}

private static Term parseTerm(String token) {
 if (isNumber(token)) return Term.constant(Double.parseDouble(token));
 return Term.variable(token);
}

private static boolean isNumber(String s) {
 try { Double.parseDouble(s); return true; }
 catch (NumberFormatException e) { return false; }
}

private double evaluateVar(String name, Map<String, Double> memo, Set<String> visiting) {
 if (memo.containsKey(name)) return memo.get(name);
 if (visiting.contains(name)) throw new IllegalStateException("Cycle detected at " + name);
 Node node = variableToNode.get(name);
 if (node == null) throw new IllegalArgumentException("Undefined variable: " + name);

 visiting.add(name);
 double result;
 if (node.kind == NodeKind.CONST) result = node.constantValue;
 else if (node.kind == NodeKind.IDENTITY) result = resolveTerm(node.left, memo, visiting);
 else {
  double leftVal = resolveTerm(node.left, memo, visiting);
  double rightVal = resolveTerm(node.right, memo, visiting);
  switch (node.op) {
   case '+': result = leftVal + rightVal; break;
   case '-': result = leftVal - rightVal; break;
   case '*': result = leftVal * rightVal; break;
   case '/': result = leftVal / rightVal; break;
   default: throw new IllegalArgumentException("Unknown op: " + node.op);
  }
 }
 visiting.remove(name);
 memo.put(name, result);
 return result;
}

private double resolveTerm(Term term, Map<String, Double> memo, Set<String> visiting) {
 if (term.isConst) return term.value;
 return evaluateVar(term.variableName, memo, visiting);
}

private enum NodeKind { CONST, IDENTITY, BINARY }

private static final class Node {
 final NodeKind kind;
 final double constantValue;
 final char op;
 final Term left, right;
 private Node(NodeKind kind, double constantValue, char op, Term left, Term right) {
  this.kind = kind; this.constantValue = constantValue; this.op = op; this.left = left; this.right
= right;
 }
 static Node constant(double v) { return new Node(NodeKind.CONST, v, 0, null, null); }
 static Node identity(String var) { return new Node(NodeKind.IDENTITY, 0, 0, Term.variable(var), n
```

```
ull); }
  static Node binary(Term l, char op, Term r) { return new Node(NodeKind.BINARY, 0, op, l, r); }
 }

 private static final class Term {
  final boolean isConst;
  final double value; final String variableName;
  private Term(boolean isConst, double value, String variableName) {
   this.isConst = isConst; this.value = value; this.variableName = variableName;
  }
  static Term constant(double v) { return new Term(true, v, null); }
  static Term variable(String n) { return new Term(false, 0, n); }
 }
}
```